

Structured Creativity: Multi-Agent Pipelines and Quality-Controlled Fiction Generation

Design Principles and Lessons from StoryMesh

Kali McLennan

May 2026

Abstract

Large language models can produce grammatically fluent, structurally coherent fiction from a single prompt call. They do this reliably, quickly, and at scale. What they do less reliably is produce fiction that feels *written* rather than generated — prose that carries the weight of human unevenness, earned specificity, and unresolved thematic pressure. This paper examines StoryMesh, a multi-stage agentic pipeline that transforms a free-text creative brief into a complete short story package including a formatted PDF, EPUB, and cover image. Rather than treating fiction generation as a single inference step, StoryMesh decomposes the task into twelve specialist stages coordinated by a LangGraph state graph, connected by Pydantic-typed contracts, and governed by a cross-provider rubric evaluation loop. The paper discusses the architectural decisions behind this decomposition, the specific mechanisms used to suppress synthetic writing patterns, and the lessons this system offers for anyone designing agentic pipelines for open-ended creative tasks. A central argument is that the most important quality intervention in a creative AI pipeline is not prompt refinement at the prose stage, but the controlled preservation of ambiguity upstream.

Contents

1	Introduction	2
2	Background and Related Work	3
2.1	Multi-Agent Orchestration for Creative Tasks	3
2.2	LLM-as-Judge, Iterative Refinement, and Critique Loops	4
2.3	The Statistical Basis of Synthetic Text	5
3	System Architecture	5
3.1	Design Philosophy	5
3.2	Pipeline Overview	6
4	Genre Understanding and Context Extraction	7
4.1	Multi-Pass Genre Normalization	7
4.2	Voice Profile Selection	8
5	Reference-Grounded Thematic Scaffolding	9
5.1	Book Fetching and Ranking	9

5.2	Theme Extraction: Framing Contradiction as Pressure	9
6	Proposal Generation and the Quality Gate	11
6.1	Craft Directives: Anti-Pattern Enforcement for Generators	11
6.2	Cross-Provider Evaluation: Why the Judge Must Be Different	12
6.3	Rubric Dimensions and Scoring	12
6.4	The Retry Loop: Critique as Creative Direction	13
7	Voice, Style, and Prose Generation	14
7.1	Voice Profiles as Prompt-Adjacent Data	14
7.2	Three-Pass Story Writing	14
8	The Near-Miss Problem: Resonance Review	15
8.1	What Near-Miss Moments Are	15
8.2	The Reviewer as a Separate Evaluation Role	15
9	Production and Delivery	16
9.1	Cover Art and Book Assembly	16
9.2	The Compare Mode: Baseline Evaluation	16
10	Design Principles for Creative AI Pipelines	17
10.1	1. Decompose by Cognitive Role, Not by Output Format	17
10.2	2. Structure Intermediate Artifacts Rigorously	17
10.3	3. Separate Generation from Evaluation Always	18
10.4	4. Feed Critique Back as Creative Direction	18
10.5	5. Preserve Ambiguity Upstream	18
10.6	6. Ground Creativity in Human Reference	19
10.7	7. Prompts Are Data, Tuning Is Configuration	19
11	Limitations and Future Directions	20
12	Conclusion	21
13	References	21

1 Introduction

In 2023, a widely circulated essay coined the term “AI slop” to describe the distinctive aesthetic of machine-generated creative writing: confident, fluent, internally consistent, and somehow hollow. The sentences were grammatical. The paragraphs advanced the plot. The prose knew what it was doing. And yet readers could feel, with uncomfortable precision, that no human intelligence had been present in the room. Something had been missing — not correctness, but the particular texture of a mind working through a problem it has not yet solved.

This is not an imaginary problem. Research in statistical text detection has shown that LLM-

generated text occupies a characteristically low-entropy region of the token distribution: models tend to choose high-probability words in high-probability combinations, producing prose that is maximally plausible and minimally surprising [Gehrmann2019]. Ironically, fluency and detectability are inversely correlated: text that humans rate as highest quality is also easiest for classifiers to identify as machine-generated [Ippolito2020]. The system optimizes for coherence, and coherence turns out to be a fingerprint.

This has practical consequences for anyone trying to build creative AI tools that produce outputs worth reading. A single-call generation approach — give the model a prompt, get a story — will reliably produce prose that is structurally sound and thematically legible and subtly wrong in a way that is hard to articulate but immediately felt. The problem is not the model’s capability. The problem is the architecture.

StoryMesh is an attempt to address this architecturally. Rather than asking a single model call to simultaneously understand genre conventions, draw on literary precedents, generate narrative seeds, draft proposals, evaluate them, write prose, and revise it, the system decomposes fiction generation into twelve specialist stages, each responsible for a narrow, well-specified task, each handing its output to the next through a Pydantic-typed contract, and each leaving a traceable artifact on disk. The result is a pipeline that takes a free-text creative brief and produces a complete short story package — formatted PDF, EPUB, and a generated cover image — in seven to ten minutes of wall-clock time.

This paper describes that pipeline: how it is designed, why it is designed that way, and what lessons it offers for practitioners building agentic systems for creative tasks. The argument is organized around three intersecting themes. First, the architecture of multi-agent creative pipelines and how role specialization produces better outputs than monolithic generation. Second, the specific mechanisms StoryMesh uses to resist the synthetic-writing failure mode — the rubric judge, the craft directives, the retry loop, and the resonance reviewer. Third, a set of generalizable design principles extracted from the system’s construction and the places where it has failed.

2 Background and Related Work

2.1 Multi-Agent Orchestration for Creative Tasks

The idea that complex generation tasks benefit from decomposition into specialist agents has been demonstrated across domains. Park et al. (2023), in their work on Generative

Agents, showed that multiple LLM instances with distinct persistent memories and social roles produce emergent, believable narrative behavior that single-instance prompting cannot replicate [park2023generative]. Wu et al. (2023) formalized this into the AutoGen framework, in which agents are defined by their system prompts and communicate through structured turns — a model that maps directly onto how StoryMesh’s LangGraph nodes relate to each other [wu2023autogen]. Hong et al. (2023), in MetaGPT, demonstrated that assigning software-engineering roles — product manager, architect, coder — to separate agents and enforcing structured document handoffs between them substantially improves output quality by preventing any single agent from holding irresolvable competing priorities [hong2023metagpt].

The application of these principles to creative writing is less studied but not absent. Mirowski et al. (2023) built Dramatron, a hierarchical pipeline for screenplay and theatre script generation that proceeds from logline through characters, plot, and finally individual scenes [mirowski2023dramatron]. Industry professionals evaluated the output; the system’s central failure mode was coherence collapse over long generation — a problem that StoryMesh’s stage-by-stage artifact persistence directly addresses. Dramatron is the closest published precursor to StoryMesh’s staged approach, though StoryMesh’s quality gate, iterative refinement loop, and voice profile system have no equivalent in that work.

2.2 LLM-as-Judge, Iterative Refinement, and Critique Loops

The idea that language models can evaluate their own outputs — and improve when given that evaluation as feedback — has produced a productive line of research. Bai et al. (2022) introduced Constitutional AI, in which Anthropic models critique their own responses against a set of explicit principles and revise accordingly [bai2022constitutional]. This is the intellectual ancestor of StoryMesh’s rubric evaluation system: the Craft Directives that govern proposal generation are evaluated by a separate agent, and the feedback is passed back as targeted revision instructions.

Shinn et al. (2023) extended this pattern in Reflexion, where agents accumulate verbal feedback across trials and store it as episodic memory, enabling genuine improvement across attempts without gradient updates [shinn2023reflexion]. The key insight Reflexion demonstrates is that a score alone is insufficient to drive improvement — the critique must be passed back as natural language context for the next generation attempt. StoryMesh’s retry loop is built on exactly this principle: when a proposal fails the rubric, the judge’s dimension-by-dimension feedback is injected verbatim into the retry prompt, giving the generator targeted creative direction rather than an opaque failure signal.

Zheng et al. (2023) provided empirical grounding for LLM-as-judge evaluation through MT-Bench and Chatbot Arena, demonstrating that strong models achieve 80%+ agreement with human raters on open-ended quality judgments, while also documenting the failure modes — positional bias, verbosity bias — that rubric-based evaluation must account for [zheng2023judging]. Their finding that cross-model agreement is higher than same-model agreement provides the empirical rationale for one of StoryMesh’s more unusual design choices: running the generation and evaluation stages on *different* LLM providers.

2.3 The Statistical Basis of Synthetic Text

Gehrmann et al. (2019) established that LLM-generated text clusters in the high-probability region of the token distribution, producing statistically “flat” distributions compared to human writing [gehrmann2019gltr]. Human writers make low-probability choices — unusual word selections, unexpected metaphors, syntactic rhythms that resist expectation — at a rate that models trained to minimize perplexity cannot replicate without deliberate intervention. This provides a theoretical basis for what practitioners observe phenomenologically: AI prose feels synthetic because it is statistically too safe.

Chakraborty et al. (2023) showed theoretically that detection becomes provably hard when generators use sampling with sufficient temperature [chakraborty2023detection]. This motivates StoryMesh’s use of non-zero sampling temperatures and voice profile overlays at the prose stage: pushing output away from the greedy-decoding attractor state is not just an aesthetic preference but a measurable distributional shift toward the region where human writing lives.

3 System Architecture

3.1 Design Philosophy

StoryMesh rests on three foundational commitments that shape every architectural decision.

Pydantic contracts at every boundary. Each pipeline stage has a defined input schema and output schema, both Pydantic models. No stage receives unstructured text from the previous stage; no stage emits unstructured text to the next. This means every inter-stage communication is typed, validated, and serializable. The constraint is sometimes awkward — certain creative outputs resist clean schema definition — but it produces two benefits that outweigh the awkwardness: it makes the system auditable (every stage output is persisted to disk as JSON),

and it makes the system robust (schema validation failures surface at the boundary rather than propagating as silent corruption into later stages).

Prompts as data, never as code. All prompt templates reside in YAML files under `src/storymesh/prompts/`. No prompt text is embedded in agent code. This means prompt engineering is a configuration task, not a software task: iterating on a prompt requires editing a YAML file, not modifying Python, running mypy, and passing tests. The system supports multiple prompt *styles* (named directories under `prompts/styles/`), each of which overrides only the files it needs to change, with the default style providing fallbacks. This architecture allows systematic experimentation — run the same brief through the default, `slim`, and `verbalized_sampling` styles and compare the outputs — without branching the codebase.

Per-stage artifact persistence. Every stage writes its output to a JSON file under `~/.storymesh/runs/<run_id>/` before the next stage begins. A run that fails at stage seven can be inspected up to stage six without re-running anything. LLM calls are logged individually to `llm_calls.jsonl`, including system prompts, user prompts, raw responses, parse success, latency, and approximate token counts. This level of observability is essential for a system doing creative work: debugging creative failures requires reading what the model was actually told, not reconstructing it from code.

3.2 Pipeline Overview

The pipeline is implemented as a LangGraph StateGraph in which each node is a Python function that reads from and writes to a shared typed state object. The state is the communication channel: nodes do not call each other directly. A stage node reads the outputs of previous stages from state, calls its agent, and writes its own output back to state. The graph topology defines the ordering.

The high-level flow is:

START

```
--> genre_normalizer
--> voice_profile_selector
--> book_fetcher
--> book_ranker
--> theme_extractor
--> proposal_draft
--> rubric_judge
    --> PASS --> story_writer --> resonance_reviewer?
```

```
--> cover_art?  
--> book_assembler --> END  
--> FAIL --> proposal_reader --> proposal_draft (revision) --> rubric_judge
```

The retry loop between `rubric_judge`, `proposal_reader`, and `proposal_draft` is the system’s primary quality-control mechanism. The `resonance_reviewer` runs only for high and `very_high` quality presets. Cover art is a no-op when no image provider is available.

A full run over a complex creative brief — with resonance review, cover art, and PDF/EPUB assembly — typically completes in seven to ten minutes of wall-clock time, executing approximately 24 LLM calls across the twelve stages. Individual stage times vary significantly with complexity: genre normalization over a well-structured prompt may complete in under fifteen seconds, while story writing and resonance review together can require four to five minutes.

4 Genre Understanding and Context Extraction

4.1 Multi-Pass Genre Normalization

Fiction generation requires understanding what kind of story the user wants before it can produce one. This sounds simple and is not. User prompts are informal creative descriptions, not taxonomic genre specifications. A prompt like “*a quiet literary mystery set in a flood-damaged coastal city*” contains an implicit genre (mystery), a tonal modifier (quiet, literary), a setting (flood-damaged coastal city), and no explicit specification of subgenres like *noir*, *psychological*, or *cozy*. A pipeline that generates the wrong kind of mystery — action-heavy where the user meant cerebral, optimistic where the user meant melancholic — has failed before the prose begins.

`GenreNormalizerAgent` addresses this through a three-pass resolution strategy. The first pass runs a static mapping against a curated lexicon of genre and tone keywords, catching exact matches at zero LLM cost. Tokens that do not match any static entry advance to a second pass: a fast, low-temperature model call that classifies remaining tokens and phrases into genres, tones, or narrative context, with explicit rules about stopwords, merged phrases, and ambiguous cases. A third pass performs holistic inference over the full original prompt, identifying genres that are *implied* by the prompt’s themes and settings but never explicitly named. This third pass is where the system catches the science-fiction signal in a prompt that only says “grimdark” and “survival,” or the horror element in a literary premise about bodily degradation.

Consider the worked example from a run whose title emerged as *The Eleven-Year Body*. The

user prompt described a collapsed future where impoverished humans consume industrial waste via metabolic drugs, slowly poisoning themselves to earn wages. The static pass resolved two explicit genre tokens: *grimdark* (mapping to fantasy) and *survival* (mapping to adventure). The holistic inference pass, operating over the full prompt text, identified three additional inferred genres: science fiction (dystopian/biopunk/cli-fi, confidence 0.90), horror (body horror/social horror, confidence 0.85), and literary fiction (social commentary/speculative, confidence 0.70). Each inferred genre included a rationale grounded in specific prompt language:

“The focus on engineered biological systems and body modification places it squarely in biopunk territory, while the environmental catastrophe and societal collapse frame it as cli-fi.”

This multi-genre portrait becomes the input to every downstream stage. The thematic extractor draws on all five genres. The rubric judge evaluates proposals against all five. The voice profile selector weighs all five when choosing a register for the prose. Normalizing genre correctly is not a preprocessing step — it is the foundation on which creative specificity is built.

4.2 Voice Profile Selection

Before the pipeline makes a single creative decision, it selects a voice profile for the run. Voice profiles are YAML data files — not code, not embedded prompts — that inject stylistic overlays into the prose generation stages. The current system ships nine profiles ranging from `literary_restraint` (“dark, restrained literary prose for mystery, noir, psychological fiction, and slow-burn dread”) to `cozy_warmth` (“warm, gentle prose for bedtime stories and quietly wondrous narratives”) to `hardboiled_fragmentary`, `documentary_collage`, `surreal_lyrical`, and others.

The `VoiceProfileSelectorAgent` matches the normalized genres, tone keywords, and raw prompt text against each profile’s keyword taxonomy, defaulting to `literary_restraint` when the signal is ambiguous. For the *Eleven-Year Body* run, despite the fantasy and adventure genre tags, the selector correctly identified that the grimdark tone and body-horror subgenres demanded restrained, observational prose:

“The prompt describes a dark, dystopian scenario with psychological and body-horror elements that demand restrained, concrete prose to convey dread and subtext. The grimdark tone and focus on systemic desperation require literary_restraint’s emphasis on detail and emotional enactment through action rather than genre_active’s kinetic momentum.”

The separation of voice from content is deliberate. The same narrative seed can be rendered in multiple registers; selecting the register early means every downstream stage writes toward a consistent stylistic target rather than discovering voice conflict at the prose stage.

5 Reference-Grounded Thematic Scaffolding

5.1 Book Fetching and Ranking

A fundamental problem with purely parametric fiction generation is that the model’s knowledge of genre conventions is statistical and blended. It knows what mystery novels do in aggregate, averaged across thousands of examples, without the friction of having read any particular one carefully. The result is proposals that feel genre-correct in a generic way — competent executions of what mystery novels do on average, rather than responses to the specific tradition a given prompt is reaching for.

BookFetcherAgent addresses this by querying Open Library for books matching the normalized genres, caching responses to avoid redundant API calls, and deduplicating by work key. BookRankerAgent then scores the fetched books using weighted signals — genre overlap, reader engagement, rating quality, and rating volume — and applies Maximal Marginal Relevance (MMR) to the final selection, trading off relevance for diversity. An optional LLM rerank pass can be enabled for higher-quality presets.

The books do not directly appear in the generated story. Their function is different: they ground the pipeline’s understanding of what this genre’s literary traditions actually are, which specific books have been canonical, and therefore what assumptions and tropes the user is implicitly drawing on when they write a particular prompt. When the *Eleven-Year Body* run fetched books for its resolved genres, it retrieved titles including *Nineteen Eighty-Four*, *Frankenstein*, *The Picture of Dorian Gray*, *The Martian*, and *Alice’s Adventures in Wonderland* — a cross-genre set that may seem eclectic but precisely captures the tradition-intersection this prompt sits at: dystopian social critique, the hubris of biological engineering, physical decay as moral register.

5.2 Theme Extraction: Framing Contradiction as Pressure

ThemeExtractorAgent is the most intellectually ambitious stage in the pipeline. It takes the genre clusters and ranked book list and produces three outputs: thematic assumptions per genre cluster, cross-cluster tensions, and narrative seeds.

The genre clusters identify the dominant tropes and thematic assumptions native to each re-

solved genre. The tension generation is where the system does something genuinely useful: it identifies *contradictions between those assumptions* and frames them as creative questions. The insight is that interesting fiction does not sit comfortably inside one genre’s logic — it lives at the friction point between two genre logics pulling in opposite directions.

Consider a run that produced the story *The Floor of the Sky*, beginning from the prompt: “A story about a desert tribe like the Fremen of Dune. The tribe knows only that they live near a wall that seems to go on forever in either direction and is impossibly tall. Surely a rebellious teenager will climb the wall or look for some other way around, over or through it. The village elders talk of the mystery of the wall, and nobody knows anything about the other side. Adventure, dark mystery, kind of 2001 Space Odyssey.” Genre normalization resolved this into mystery, science fiction, adventure, and fantasy — an unusual four-way intersection. The system generated five tensions across those genre clusters. Two illustrate the mechanism:

T1 — Mystery (“Sacred, unexplained phenomena are the engine of meaning and identity”) vs. *Science Fiction* (“All systems can be understood, mapped, and potentially mastered”): **Creative question:** “What happens to a person — and a culture — when the sacred mystery they have organized their entire existence around turns out to have a mundane, engineered explanation? Is the truth a liberation or a bereavement?”

T3 — Fantasy (“The world contains forces larger than human institutions, and these forces ultimately arbitrate justice”) vs. *Science Fiction* (“The consequences of scientific or social engineering are the proper subject of moral scrutiny”): **Creative question:** “If the village elders’ myths about the wall are simultaneously true as lived meaning and false as literal description of reality, what is the status of their authority — and what does the teenager owe them?”

Each tension also receives a list of *clichéd resolutions* — specific, named story moves that would resolve the tension cheaply. These cliché lists are propagated forward to the rubric judge, which penalizes proposals that fall into them. For T1, the flagged clichés include: “The teenager climbs the wall, discovers it is a constructed habitat or simulation, and returns to ‘free’ the tribe with the truth — which they gratefully accept” and “A scientist or engineer character appears on the other side who has all the answers and delivers them in a single expository scene.” Naming these in advance forces both the proposal generator and the evaluator to recognize them as paths to avoid.

From the tensions, the extractor generates five narrative seeds — character-centered story concepts that inhabit specific tensions without resolving them. Seed S1, ultimately selected for *The*

Floor of the Sky, introduces Sura: a seventeen-year-old who has memorized every elder-song about the wall not out of reverence but to find the inconsistencies, and who, when she finally scales it, does not find another desert on the other side but a curved interior surface descending into darkness. *“The tribe has been living inside something. The question is whether it is a vessel, a prison, or a god’s unfinished thought — and whether the builders are still aboard.”*

This stage establishes the conditions under which a human-feeling story can emerge. The tensions and seeds are not instructions to the story writer; they are a substrate of unresolved contradiction for it to inhabit.

6 Proposal Generation and the Quality Gate

6.1 Craft Directives: Anti-Pattern Enforcement for Generators

The `ProposalDraftAgent` takes an assigned seed, the thematic tensions, the clichéd resolutions, and a genre portrait, and generates three candidate story proposals before self-selecting the strongest. The key innovation at this stage is the Craft Directives system: a set of explicit, verifiable instructions that suppress the specific patterns making LLM-generated fiction feel synthetic.

The directives are implemented as a labeled system in the generation prompt (CD-1 through CD-8), each with a priority level (CRITICAL, HIGH, MEDIUM, LOW). Examples:

- **CD-1 [CRITICAL] — No Tension Resolution:** *“The central thematic tension must remain productively unresolved at the end of the story. Both sides of the tension must remain true simultaneously.”* With a concrete violation example: a romance/horror story where love defeats the monster, resolving the tension by selecting one genre’s worldview over the other.
- **CD-2 [CRITICAL] — Specificity over Abstraction:** Every character trait and plot mechanism must be concrete and particular to this story. *“NOT ‘a grizzled detective’ – instead: ‘a census-taker who memorized the population of every district before the collapse.’”*
- **CD-6 [MEDIUM] — Anti-Pattern Avoidance:** An explicit enumeration of LLM writing tics: characters whose eyes “reflect,” “mirror,” or “hold” anything; weather used to signal mood; protagonists who “realize” or have an “epiphany” in the climax; the construction “In a world where...”; describing anything as “a dance” or “a tapestry” metaphorically.

The CD system embodies a key insight: prohibiting patterns explicitly is more effective than instructing toward their alternatives. Telling a model to write “with unexpected specificity” produces incremental improvement. Telling it that a specific set of named patterns will cause rejection produces categorical avoidance.

6.2 Cross-Provider Evaluation: Why the Judge Must Be Different

RubricJudgeAgent evaluates the selected proposal against the CD system and the thematic tensions. The most structurally unusual feature of this stage is that it runs on a *different LLM provider* than the generation stage. When ProposalDraftAgent uses an Anthropic model, RubricJudgeAgent uses an OpenAI model, and vice versa.

The rationale was documented in the system’s planning notes:

“If both use the same LLM, the evaluator inherits the generator’s blind spots — it tends to rate its own model’s output favorably and is less likely to catch stylistic tics or structural laziness that are characteristic of that model.”

This design is supported by Zheng et al. (2023), whose MT-Bench work found that cross-model agreement on quality judgments was higher than same-model self-agreement, and that model self-evaluation shows consistent positional and verbosity biases. An OpenAI judge evaluating an Anthropic-generated proposal brings genuinely independent editorial perspective. The provider registry and per-agent configuration already support this — it is a configuration-level choice with no code change required.

6.3 Rubric Dimensions and Scoring

The rubric evaluates proposals on five dimensions, each scored 0–2 and carrying different weight in the composite:

Dimension	What It Measures	Weight
protagonist_interiority	Specificity of the character’s want/need split to <i>this</i> story	High
restraint	Absence of over-explanation; implication over statement	High
story_serving_choices	Whether plot elements are surprising yet inevitable	High
user_intent_fidelity	Alignment with the user’s stated genres, tones, context	Medium

Dimension	What It Measures	Weight
specificity	Density of concrete, non-fungible detail	Medium

Each dimension produces a score and actionable feedback. The composite score is the sum of dimension scores; a configurable threshold determines pass or fail. Clichéd resolutions carry a hard penalty: each matched item from the tensions’ cliché lists reduces the composite score.

The scoring is computed by the Python agent, not the LLM. The model returns scores and feedback text; the agent performs the weighted aggregation and threshold comparison. This keeps pass/fail deterministic and auditable regardless of how the evaluating model might characterize the overall quality.

6.4 The Retry Loop: Critique as Creative Direction

When a proposal fails, the pipeline does not immediately escalate to the next attempt with the same prompt. Instead, `ProposalReaderAgent` synthesizes the rubric feedback into editorial guidance, and `ProposalDraftAgent` receives both the failed proposal and the dimension-specific critique in its retry prompt:

“This is attempt N. A previous proposal was evaluated by an independent editorial judge and did not pass. Your job is to generate a NEW proposal that addresses the identified weaknesses while preserving any strengths noted. Do NOT simply patch the previous proposal — generate fresh creative work that takes the feedback as creative direction.”

This design echoes Reflexion’s finding that verbal critique passed as generation context produces meaningful improvement across attempts. The system tracks proposal history and rubric history across all retry attempts and selects the best-scoring proposal when the retry budget is exhausted, ensuring that maximum-retry failures still produce the strongest available result.

The *Eleven-Year Body* run required four rubric evaluation attempts before progression to story writing. The final rubric output (on the attempt that cleared the threshold) identified the story’s main strength — “the setting of The Narrows, which is vividly described and adds a tangible sense of place” — and its persistent weakness: *“The thematic thesis over-explains the story’s meaning rather than letting it emerge naturally from the narrative.”*

This failure mode — over-explanation of meaning — is precisely what the prose pipeline redesign work has identified as the deepest structural problem in AI-generated literary fiction. The rubric catching it at the proposal stage, before prose generation begins, is the quality gate working as

designed.

7 Voice, Style, and Prose Generation

7.1 Voice Profiles as Prompt-Adjacent Data

Voice profiles are YAML files, not code, and they are injected into system prompts at runtime rather than embedded in any agent implementation. A profile can override the following:

- **craft overlay:** positive stylistic instructions injected into the story writer’s system prompt
- **avoid overlay:** explicit prohibitions specific to this voice
- **summary overlay:** instructions for the back-cover summary pass
- **exemplars:** short prose samples that model the target register

This architecture separates the voice selection decision from the prose generation mechanism. The StoryWriterAgent does not know which voice profile it is running under — it only sees the injected text. Swapping profiles requires no code changes. Adding a new profile requires creating a single YAML file.

7.2 Three-Pass Story Writing

StoryWriterAgent generates fiction in three sequential passes:

1. **Scene outline:** A structured plan of scenes, each with a concrete observational anchor (an object, a procedure, a sensory detail that the prose will return to), a statement of narrative pressure rather than thematic function, and an explicit flexibility note giving the prose model permission to linger, compress, or add local detail.
2. **Full draft:** The prose itself, generated from the scene outline with voice-profile overlays active. The draft prompt explicitly discourages the model from summarizing, over-explaining, or closing tensions cleanly.
3. **Back-cover summary:** A condensed synopsis of the completed story, written after the draft is available.

The separation of outline from draft is significant. Without it, the model generating prose is simultaneously the model deciding scene structure, thematic emphasis, and narrative pacing — a situation analogous to a writer trying to architect and draft at the same moment. The outline pass crystallizes structure while leaving the prose stage free to make local discoveries.

One identified failure pattern at the outline stage is what the system’s design notes call *premature interpretation*: scene summaries that accurately describe what happens but also embed a complete analytical reading of the scene’s meaning, leaving the prose stage no interpretive work to do. The prompt engineering at this stage attempts to keep scene descriptions behavioral and physical rather than analytical.

8 The Near-Miss Problem: Resonance Review

8.1 What Near-Miss Moments Are

The most subtle quality problem in the system is not what fails outright but what almost succeeds. The ResonanceReviewerAgent exists to address a pattern that emerged from systematic examination of the pipeline’s outputs: moments where the story introduces genuine emotional or philosophical stakes and then moves past them without engaging with the implications.

The system’s planning documentation describes this as the “composure over pressure” failure mode:

“The pipeline tends to: generate structurally sound and thematically suggestive setups; maintain stylistic restraint and avoid over-explanation; move forward once a meaningful implication is introduced. This results in prose that feels intelligent and controlled, contains depth, but does not always engage with that depth.”

These near-miss moments — points where the story brushes against deeper meaning but retreats — are not failures of idea generation. They are failures of follow-through. They emerge from competing optimization pressures that are each individually correct: avoid over-explaining, maintain subtlety, preserve structural clarity, prevent melodrama. Together, these constraints create a systematic bias toward stabilizing volatile moments rather than inhabiting them.

8.2 The Reviewer as a Separate Evaluation Role

The resonance reviewer runs a post-hoc analysis of the completed draft, identifying two or three near-miss moments and expanding them in-place — adding pressure without resolving the underlying tension, dwelling rather than moving past. Critically, it does not rewrite the entire story: it intervenes locally, at identified moments, and then regenerates the back-cover summary from the revised draft.

The separation of reviewer from writer is the key structural choice. The writing agent, already

optimizing for coherence and structural completion, is poorly positioned to identify its own under-explored moments — it has already decided that the moment was handled sufficiently. A separate stage operating at the meta level can compare what the story *implies* against what it *explores* and identify the gap.

This mirrors the function of a human editor in a writing workshop: the author has internalized the story and cannot easily see what a fresh reader experiences. The reviewer has no investment in the existing draft and can ask, with genuine curiosity, what the story was almost saying.

The resonance reviewer runs only for high and very_high quality presets, given its computational cost (roughly 100 additional seconds of generation time). For draft and standard presets, the cost-quality tradeoff favors omission.

9 Production and Delivery

9.1 Cover Art and Book Assembly

CoverArtAgent takes the selected proposal’s `image_prompt` field — generated as part of the proposal drafting process — and appends a flat-canvas enforcement suffix before submitting to the image generation backend. The generated image is then composited with the story title and byline using Pillow, producing a cover image that is included in the assembled book artifacts.

BookAssemblerAgent produces a formatted PDF and EPUB from the story content, with the cover image, title page, and back-cover summary assembled into publication-ready output. The assembly writes to the run directory alongside all other stage artifacts.

The pipeline also supports direct email delivery via SMTP: running with `--email` sends the assembled PDF and EPUB to the specified address when assembly completes. In kiosk mode — a React frontend backed by a FastAPI server with Server-Sent Events for live progress updates — email submission goes through an environment variable to preserve privacy (the email is never passed through command-line arguments visible to logging).

9.2 The Compare Mode: Baseline Evaluation

A dedicated compare command runs a full StoryMesh pipeline and then generates a single-call baseline using the same user prompt, the same word count target, and the story writer’s model. Both outputs are assembled into a blinded evaluation packet: two anonymized candidates (A and B), with the key identifying which is which stored separately.

This design supports both human blind review and LLM-as-judge workflows. The blinded packet contains the original user prompt, both full story drafts, and the word count of each candidate. Neither candidate’s provenance is visible in the packet. A reviewer — human or model — can evaluate on craft alone.

The compare mode represents a starting point for systematic quality measurement. The system does not yet aggregate blinded evaluation results into a score; that capability is identified as future work.

10 Design Principles for Creative AI Pipelines

The construction of StoryMesh, including its failures and the revisions those failures motivated, suggests a set of generalizable principles for anyone designing multi-stage agentic systems for creative tasks.

10.1 1. Decompose by Cognitive Role, Not by Output Format

The instinctive decomposition for a fiction pipeline is structural: *“generate an outline, then generate scenes, then generate prose.”* This is output decomposition — it divides the task by what comes out of each stage. A better decomposition divides by what cognitive mode each stage requires.

Genre normalization requires taxonomic classification. Theme extraction requires literary comparative analysis. Proposal generation requires constrained creative invention. Rubric evaluation requires dispassionate editorial judgment. Story writing requires observed, embodied scene rendering. Resonance review requires meta-level comparison between implication and execution.

These are genuinely different cognitive modes, and a model switching between them within a single prompt is under-serving all of them simultaneously. Assigning each to a separate agent — with its own system prompt, its own temperature setting, its own provider if warranted — allows each to be optimized independently.

10.2 2. Structure Intermediate Artifacts Rigorously

Every inter-stage communication in StoryMesh is a Pydantic model. This means intermediate artifacts are not narrative summaries (“write a two-paragraph description of the themes for the next stage”) but structured data with typed fields, validation constraints, and schema versions.

The practical value of this is substantial. Thematic tensions, narrative seeds, and clichéd resolutions are not prose — they are structured objects that the rubric judge can query, the proposal agent can iterate over, and the story writer can render against. The cliché avoidance system depends entirely on this: if the clichéd resolutions were embedded in narrative summary text, the downstream stages could not check proposals against them mechanically.

Schema versioning matters for a system producing artifacts that may be compared or replayed weeks after generation. A `schema_version` field on every output ensures that inspection tools and evaluation pipelines can handle the evolution of field definitions without silent errors.

10.3 3. Separate Generation from Evaluation Always

No stage in StoryMesh evaluates its own output for pipeline purposes. The proposal generator has an internal self-selection step (choosing among N candidates), but the external quality gate is a separate agent on a separate provider. The story writer does not assess whether its draft has near-miss moments — a separate reviewer does that.

This principle costs computational resources. It costs time. It is worth it. The Zheng et al. (2023) finding on cross-model agreement is the empirical basis; the intuition is that any agent optimizing for generation cannot simultaneously optimize for evaluation — the two tasks pull in opposite directions. Evaluation requires stepping outside the generative process to ask whether the output is *what the process should have produced*, which is a question the generative process cannot pose to itself without circularity.

10.4 4. Feed Critique Back as Creative Direction

When the rubric judge fails a proposal, the failure signal that reaches the next attempt is not a score — it is dimension-by-dimension prose feedback injected into the generator’s context. The generator on the retry is not told it scored 5/10; it is told, specifically, that its thematic thesis was over-explained and that its plot arc took a predictable path despite the original tension. This is actionable creative direction.

This design reflects Reflexion’s finding that verbal critique passed as context enables improvement that score signals alone do not. A score tells the generator it failed; a critique tells the generator how to fail differently. Only the latter enables the pipeline to improve.

10.5 5. Preserve Ambiguity Upstream

Perhaps the most counterintuitive lesson from StoryMesh’s design iterations is that the most important quality intervention does not happen at the prose stage. It happens earlier, in how

the proposal is structured and what the outline is allowed to claim.

The core problem identified in the system’s prose pipeline redesign work is premature closure: the proposal articulates the story’s philosophical meaning as a clean thesis before the prose has begun; the outline encodes a complete scene-level interpretation before the prose has been written; the draft model therefore functions as a renderer of pre-decided meaning rather than a discoverer of lived-in meaning. The result is prose that is polished, structurally sound, and slightly too engineered.

The fix is not to make the prose prompt longer or more elaborate. The fix is to protect ambiguity upstream: replace `thematic_thesis` (a clean answer) with `thematic_pressure` (an unresolved question), add `unknowns` fields to proposals, and restrict outline scene descriptions to behavioral and physical content rather than interpretive analysis. When the prose stage inherits material that is still partly open, it can write as if discovering the story rather than executing it.

10.6 6. Ground Creativity in Human Reference

BookFetcherAgent and BookRankerAgent are the pipeline’s ground-truth connection to the literary tradition a prompt is reaching for. The model’s parametric knowledge of genre is statistical and blended. Fetching real books — books that readers have rated, that critics have placed in traditions, that library systems have classified — provides a concrete reference set for the theme extraction stage.

The result is not citation or imitation. No fetched book appears in the generated story. The result is that the thematic assumptions and dominant tropes identified by the theme extractor are grounded in what those books actually do, not in the model’s aggregate impression of what the genre tends to do. The difference is audible in the specificity of the tensions generated.

10.7 7. Prompts Are Data, Tuning Is Configuration

The craft directives, rubric dimensions, voice overlays, and prompt styles are all data files. This is not merely an organizational preference — it defines what kind of work quality improvement requires. When a craft directive is too strong and producing stilted proposals, editing a YAML file and running the pipeline is the work. When a new anti-pattern emerges in the output, adding it to the CD-6 list is the work. No code review, no test suite, no deployment.

This principle is in tension with good software engineering: prompt text in data files is harder to test statically than typed code. StoryMesh addresses this through prompt-loading integration tests that verify placeholder presence and format correctness, and through the observe-edit-rerun workflow that the artifact persistence system enables. The tradeoff is worth it for a system

whose most important engineering happens in natural language.

11 Limitations and Future Directions

StoryMesh is a working system, not a finished one. Several limitations are worth stating directly.

Provider dependency. Many stages degrade gracefully when provider keys are absent, but graceful degradation is not the same as full capability. Cover art requires an image provider. Rubric evaluation on a cross-provider basis requires both providers to be available. A user with only a single provider configured will not get the full benefit of the cross-model evaluation design.

Rubric bias toward literary sensibility. The rubric as currently tuned rewards restraint, specificity, and unresolved tension — a sensibility that belongs to literary fiction and quiet horror more than to commercial genre fiction or high-energy adventure. A cozy mystery novel that produces clean emotional resolution is not a failure; a commercial thriller that moves quickly and explains itself is not inferior. The rubric should be configurable by genre, not universal.

The near-miss detection problem is partially solved. The resonance reviewer can identify passages where the draft retreats from emotional pressure, but its ability to expand those moments without disrupting narrative continuity is imperfect. The risk of over-expansion — adding explicit depth that destroys the implication it was meant to honor — remains real and is monitored only through human inspection of the output.

Systematic evaluation is nascent. The compare mode and blinded evaluation packet provide the infrastructure for measuring whether the multi-stage pipeline outperforms single-call generation, but the system does not yet aggregate those evaluations into a score that tracks quality across runs. That capability is the most important near-term work.

The “human-feeling” problem is partially addressed. The distributional argument — that voice profiles and non-greedy sampling push output toward the human-text region — is theoretically grounded but not empirically measured within StoryMesh. Whether the system’s outputs are distinguishable from human-written short fiction at the same length and genre is an open question that the evaluation infrastructure, once built out, could begin to answer.

12 Conclusion

The central bet StoryMesh makes is that generating quality fiction is a decomposition problem before it is a capability problem. Large language models already have the knowledge base to produce compelling characters, specific world-building details, and prose that enacts rather than explains. What single-call generation cannot provide is the structure needed to direct that knowledge without over-constraining it: to understand genre without resolving it into generic competence, to generate creative pressure from the friction between traditions, to evaluate output against principled criteria without that evaluation being captured by the generator’s own biases, and to identify where meaning is being implied but not inhabited.

The twelve stages of the pipeline are not twelve ways of doing what one stage could do — they are twelve genuinely distinct cognitive operations, each of which is necessary and none of which the others can absorb without losing something essential.

The most important lesson from building this system is also the least intuitive: the right place to intervene for prose quality is not the prose prompt. It is the proposal structure, the outline philosophy, and the rubric dimensions. By the time the story writer sees its input, the creative decisions that will determine whether the output feels written or generated have already been made. The system that produces human-feeling fiction is not the one with the most elaborate prose instructions — it is the one that most carefully preserves ambiguity for the model to discover.

13 References

- Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique Through Self-Reflection. *ICLR 2024*.
- Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., ... & Kaplan, J. (2022). Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*. Anthropic.
- Chakraborty, S., Bedi, A. S., Zhu, S., An, B., Manocha, D., & Huang, F. (2023). On the Possibilities of AI-Generated Text Detection. *arXiv:2304.04736*.
- Gehrmann, S., Strobelt, H., & Rush, A. M. (2019). GLTR: Statistical Detection and Visualization of Generated Text. *ACL 2019 System Demonstrations*.
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., ... & Wu, Q. (2023). MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *arXiv:2308.00352*.

- Ippolito, D., Duckworth, D., Callison-Burch, C., & Eck, D. (2020). Automatic Detection of Generated Text is Easiest When Humans are Fooled. *ACL 2020*.
- Mirowski, P., Mathewson, K. W., Pittman, J., & Evans, R. (2023). Co-Writing Screenplays and Theatre Scripts with a Language Model: Evaluation by Industry Professionals. *ACL 2023*.
- Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *UIST 2023*.
- Shinn, N., Cassano, F., Labash, B., Gopalan, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS 2023*.
- Uchendu, A., Ma, Z., Le, T., Zhang, R., & Lee, D. (2021). TURINGBENCH: A Benchmark Environment for Turing Test Studies. *EMNLP 2021 Findings*.
- Veselovsky, V., Ribeiro, M. H., & West, R. (2023). Artificial Artificial Artificial Intelligence: Crowd Workers Widely Use Large Language Models for Text Production Tasks. *arXiv:2306.07899*.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., ... & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv:2308.08155*. Microsoft Research.
- Zheng, L., Chiang, W. L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., ... & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS 2023 Datasets and Benchmarks*.